

Cuadro comparativo / Matriz de costo

Matriz de decisión de modelo de IA por tarea + simulación de ahorro — Sistema "Atención consultas compras y alquileres Inmobiliaria IA AUTOMATION" (n8n + OpenAI + Notion)

Este documento justifica qué modelo/estrategia de procesamiento se usa en cada tarea del flujo real construido en n8n, y cuantifica el ahorro de las optimizaciones aplicadas. No es solo "limitar tokens", es una decisión de arquitectura por tarea, tomando como base la configuración real del nodo OpenAI Chat Model del flujo (modelo gpt-4o-mini, maxTokens = 800, Response Format: json_object).

1. Matriz de decisión: modelo / estrategia por tarea

Tarea del flujo	Modelo / Estrategia elegida	Complejidad	Por qué esta elección
Triage en tiempo real: clasificar intención, extraer código de propiedad, calcular prioridad sugerida y redactar la respuesta al cliente (nodo AI Agent, Circuito 1)	GPT-4o-mini · modo estándar (sin Batch)	Baja-media: clasificación estructurada + texto corto, sin razonamiento profundo ni documentos extensos	Es una tarea acotada y repetitiva (un correo corto por vez). Un modelo mayor (GPT-4o, Claude) no mejora la calidad de forma perceptible acá y sí encarece ~10-15x el costo por token. Además el cliente espera respuesta en minutos, así que Batch API (demora de hasta 24 hs) queda descartado para esta tarea.
Consulta al catálogo de propiedades (Notion Tool: buscar_propiedad / listar_propiedades, usadas como tools del AI Agent)	Sin modelo de IA — llamada directa a la API de Notion (filtro exacto)	N/A	Es una búsqueda estructurada por código u operación exacta. Resolverla con un modelo de lenguaje sería pagar tokens por algo que una query determinística resuelve gratis y sin margen de error.
Recalculo de PRIORIDAD, saneo del JSON y verificación anti-alucinación de codigo_valido (nodo Code · JavaScript)	Sin modelo de IA — JavaScript determinístico	N/A	Confiarle a la IA el cálculo de reglas de negocio (o pedirle que revise su propia salida) implicaría una segunda llamada a la API — el doble de costo — y además es menos confiable que una regla de código fija. Se resuelve gratis y 100% verificable.
Aprobación humana + notificación multicanal: Slack, Telegram, Gmail Reply (Circuito 2 completo)	Sin modelo de IA	N/A	Son acciones mecánicas sobre datos que la IA ya procesó en el Circuito 1 (no se genera texto nuevo). Ejecutarlas sin IA es la opción de costo cero.
System message del agente (reglas de negocio + descripción de tools, ~800 tokens estáticos, idéntico en cada corrida)	Prompt Caching automático de OpenAI	—	OpenAI cachea automáticamente el prefijo del prompt cuando supera ~1024 tokens y es idéntico entre llamadas consecutivas (no requiere configuración extra en n8n). Como el system message no cambia nunca entre correos, esos tokens de entrada se cobran a mitad de precio en cada llamada siguiente.

Tarea del flujo	Modelo / Estrategia elegida	Complejidad	Por qué esta elección
(Extensión futura — no implementada hoy) Reprocesamiento masivo del catálogo, ej. regenerar descripciones de todas las propiedades tras una actualización de criterios	Batch API (candidata, si se implementara)	Alto volumen, baja urgencia	Es la única tarea de este sistema que calzaría con Batch: no depende de un cliente esperando en el momento. Hoy no está implementada porque el catálogo no se recarga de forma masiva, pero queda documentada como la ruta correcta si el volumen crece.

2. Simulación de ahorro

Supuestos de la simulación (para que el cálculo sea auditable, no una cifra inventada): una inmobiliaria con 10.000 consultas de clientes por mes (escenario de alto volumen). Por llamada al AI Agent: ≈ 800 tokens de entrada del system message (estático) + ≈ 150 tokens del correo del cliente (variable) = 950 tokens de entrada; ≈ 250 tokens de salida en promedio (JSON compacto, tope configurado de 800 vía "maxTokens"). Precios usados por millón de tokens: GPT-4o-mini estándar = \$0.15 / 1M tokens de entrada y \$0.60 / 1M de salida; con Prompt Caching, el tramo cacheado de entrada baja a \$0.075 / 1M (precios publicados por OpenAI).

Estrategia de procesamiento	Costo simulado / mes	% de ahorro	Explicación técnica y financiera
1. Estándar (sin optimizar) — system message completo repetido en cada llamada, sin caching	\$2.93 USD	0%	9,5 M tokens de entrada $\times$ \$0.15/1M (\$1.43) + 2,5 M tokens de salida $\times$ \$0.60/1M (\$1.50). Se paga el system message completo (≈ 800 tokens) en cada una de las 10.000 consultas, sin ningún descuento.
2. GPT-4o-mini + Prompt Caching automático (esto es lo que corre hoy en el flujo, sin configuración adicional en n8n)	\$2.33 USD	$\approx 20\%$	8 M tokens del system message cacheado $\times$ \$0.075/1M (\$0.60) + 1,5 M tokens del texto del correo (no cacheable, varía) $\times$ \$0.15/1M (\$0.225) + salida sin cambios (\$1.50). El ahorro es real pero parcial: el caching solo afecta a los tokens de entrada repetidos, no a la salida (que es la mitad del costo total).
3. + Batch API (hipotético — aplicaría solo a una tarea masiva y no urgente, no al triage en tiempo real)	$\approx$\$1.16 USD	$\approx 60\%$	Combina el 50% de descuento por lote (sobre entrada y salida) con el caching. Es la cifra más baja, pero requeriría tolerar hasta 24 hs de demora — inviable para responder correos de clientes en tiempo real, que es el objetivo del sistema. Por eso no se aplica hoy.